

AutoMapper Ignore Method in C# with Examples

In this article, I will discuss using the **AutoMapper Ignore Method in C#** with Examples. Please read our previous article discussing [AutoMapper Conditional Mapping in C#](#) with Examples. At the end of this article, you will understand the need and use of the AutoMapper Ignore Method in C# with Examples.

Why do we need the AutoMapper Ignore Method in C#?

AutoMapper, a popular object-to-object mapping library in C#, simplifies the task of transforming objects from one type to another. When configuring AutoMapper mappings, you might encounter scenarios where you do not want to map certain properties. In these cases, the Ignore() method is used to ignore specific properties during the mapping process.

By default, AutoMapper tries to map all the properties from the source to the destination type when the names of both the source and destination type are the same. If you want some properties not to map with the destination type property, you must use the AutoMapper Ignore Method in C#.

Example to understand AutoMapper Ignore Method in C#

Let us understand how to use the AutoMapper Ignore Method with an example. We will use the following **Employee** and **EmployeeDTO** classes: AutoMapper Ignore Property. Both classes have the same number, same name, and same types of properties. So, create a class file named Employee.cs and copy and paste the following code.

```
namespace AutoMapperDemo
{
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public string Address { get; set; }
        public string Email { get; set; }
    }
}
```

Next, create another class file with the name EmployeeDTO.cs and then copy and paste the following code into it.

```
namespace AutoMapperDemo
{
    public class EmployeeDTO
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public string Address { get; set; }
        public string Email { get; set; }
    }
}
```

Business Requirement:

Our Business Requirement is not to map the **Address Property**, i.e., we need to **Ignore the Address Property** while mapping between these objects. To do so, we need to use the **Ignore Method** with the **Address** property of the destination type while doing the mapper configuration. So, create a class file named MapperConfig.cs and copy and paste the following code. Here, using the ForMember method, we are calling the Ignore method for the destination Address property, which will Ignore this property while doing the Mapping.

```
using AutoMapper;
namespace AutoMapperDemo
{
    public class MapperConfig
    {
        public static Mapper InitializeAutomapper()
        {
            //Configuring AutoMapper
            var config = new MapperConfiguration(cfg =>
            {
                cfg.CreateMap<Employee, EmployeeDTO>()
                    //Ignoring the Address property of the destination type
                    .ForMember(dest => dest.Address, act => act.Ignore());
            });
            //Creating the Mapper Instance
            var mapper = new Mapper(config);
            //returning the Mapper Instance
            return mapper;
        }
    }
}
```

Next, modify the Main method of the Program class as follows to see whether the Ignore Method of AutoMapper is working as expected or not.

```
using System;
namespace AutoMapperDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Initialize AutoMapper
            var mapper = MapperConfig.InitializeAutomapper();
            //Creating Source Object
            Employee employee = new Employee()
            {
                ID = 101,
                Name = "James",
                Address = "Mumbai",
                Email = "info@dotnettutorials.net"
            }
        }
    }
}
```

```

    };
    //Mapping Source Employee Object with Destination EmployeeDTO Object
    var empDTO = mapper.Map<Employee, EmployeeDTO>(employee);
    //Printing the Employee Object
    Console.WriteLine("After Mapping : Employee Object");
    Console.WriteLine("ID : " + employee.ID + ", Name : " + employee.Name + ", Address : " + employee.Address + ", Email : " + employee.Email);
    Console.WriteLine();
    //Printing the EmployeeDTO Object
    Console.WriteLine("After Mapping : EmployeeDTO Object");
    Console.WriteLine("ID : " + empDTO.ID + ", Name : " + empDTO.Name + ", Address : " + empDTO.Address + ", Email : " + empDTO.Email);
    Console.ReadLine();
}
}
}

```

Now, run the application, and you will get the following output.

```

After Mapping : Employee Object
ID : 101, Name : James, Address : Mumbai, Email : info@dotnettutorials.net

After Mapping : EmployeeDTO Object
ID : 101, Name : James, Address : , Email : info@dotnettutorials.net

```

As you can see in the above output, the value for the Address property is empty even though the Address property for the Source type has value. So, the AutoMapper Ignore() method is used when you want to ignore the property in the mapping completely. The ignored method could be in either the source or the destination object.

How to Ignore Multiple Properties while using AutoMapper in C#?

In our previous example, we have seen how to use the Ignore Method with one Property. This is fine if you are Ignoring one or two properties. But, it will become tedious if you want to ignore multiple properties from mapping. If that is your requirement, consider creating an extension class that will Ignore the properties based on the data attribute specified in the model. Let us discuss the step-by-step procedure to implement this.

Step1: Create a NoMap Attribute Class

Create a class file named **NoMapAttribute.cs** and copy and paste the following code. Here, you can see the NoMapAttribute class is inherited from the System.Attribute class. Now, we can use this class as an attribute.

```

namespace AutoMapperDemo
{
    public class NoMapAttribute : System.Attribute
    {
    }
}

```

Step2: Decorate the properties with the NoMap attribute

We need to decorate the **NoMap** Attribute with the source type properties we don't want to map with the destination type. For example, if you don't want to map the Address and Email property, then you need to decorate the NoMap attribute with the Address and Email property of the Employee class. So, modify the Employee class as follows.

```
namespace AutoMapperDemo
{
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        [NoMap]
        public string Address { get; set; }
        [NoMap]
        public string Email { get; set; }
    }
}
```

Step3: Creating an Extension Method

Create a class file named IgnoreNoMapExtensions.cs, and then copy and paste the following code. This is going to be a static class, and within this static class, we are creating one generic static method and passing the parameter as IMappingExpression using this keyword. Within this method, we check whether the property is decorated with the NoMap Attribute. If it is decorated with NoMap Attribute, we call the Ignore method.

```
using AutoMapper;
using System.ComponentModel;
namespace AutoMapperDemo
{
    //Extension Method:
    //The Class Should Be Static
    //Method Should Be Static
    //First Parameter is the class which which we can access the Method
    public static class IgnoreNoMapExtensions
    {
        //Method is Generic and Hence we can use with any TSource and TDestination
        public static IMappingExpression<TSource, TDestination> IgnoreNoMap<TSource, TDestination>(
            this IMappingExpression<TSource, TDestination> expression)
        {
            //Fetching Type of the TSource
            var sourceType = typeof(TSource);
            //Fetching All Properties of the Source Type using GetProperties()
            foreach (var property in sourceType.GetProperties())
            {
                //Get the Property Name
            }
        }
    }
}
```

```

       PropertyDescriptor descriptor = TypeDescriptor.GetProperties(s
[property.Name]);
        //Check if Property is Decorated with the NoMapAttribute
        NoMapAttribute attribute =
        (NoMapAttribute)descriptor.Attributes[typeof(NoMapAttribute)];
        if (attribute != null)
        {
            //If Property is Decorated with NoMap Attribute, call the
Method
            expression.ForMember(property.Name, opt => opt.Ignore());
        }
    }
    return expression;
}
}
}

```

Step4: Using the IgnoreNoMap Extension Method in Mapper Configuration

In the last step, we need to use the IgnoreNoMap method while defining the Mapping. So, modify the InitializeAutomapper method of the MapperConfig class as follows. Here, you can see after the Mapping, we are simply calling the IgnoreNoMap method. This IgnoreNoMap Extension method is generic and can be used with any other mapping if required.

```

using AutoMapper;
namespace AutoMapperDemo
{
    public class MapperConfig
    {
        public static Mapper InitializeAutomapper()
        {
            //Configuring AutoMapper
            var config = new MapperConfiguration(cfg =>
            {
                cfg.CreateMap<Employee, EmployeeDTO>()
                    .IgnoreNoMap();
            });
            //Creating the Mapper Instance
            var mapper = new Mapper(config);
            //returning the Mapper Instance
            return mapper;
        }
    }
}

```

With the above changes in place, run the application, and it will display the following output. You can see it is Ignoring the Address and Email Property values.

```
After Mapping : Employee Object  
ID : 101, Name : James, Address : Mumbai, Email : info@dotnettutorials.net  
  
After Mapping : EmployeeDTO Object  
ID : 101, Name : James, Address : , Email :
```

Use Cases for Ignore Method

- **Property Mismatches:** When the source and destination types do not have the same properties, you want to prevent AutoMapper from trying to map these mismatched properties.
- **Read-Only Properties:** If the destination type has read-only properties, that should not be modified during mapping.
- **Conditional Mapping:** In scenarios where certain properties should be mapped or ignored based on specific conditions.
- **Performance Optimization:** Ignoring properties that are expensive to map or are not needed in the destination object can improve performance.

In the next article, I will discuss [How to Store Fixed and Dynamic Values in Destination Property in AutoMapper with Examples in C#](#). In this article, I explain the **AutoMapper Ignore Method in C#** with Examples. I hope this AutoMapper Ignore Method in C# article will help you with your needs. I would like to have your feedback. Please post your feedback, questions, or comments about this article.